NumPy Cheat Sheet

NumPy 2.5 · Python ≥ 3.12 (examples on 3.13)
Checked October 2026 · albertoirurueta.github.io/docs

Creation

```
import numpy as np
np.array([[1, 2], [3, 4]])      # from lists
np.zeros((2, 3)) ; np.ones(4) ; np.empty(3)
np.full((2, 2), 7) ; np.eye(3)
np.arange(0, 10, 2)             # step, half-open
np.linspace(0, 1, 5)            # count, inclusive end
np.zeros_like(a) ; np.ones_like(a)
xx, yy = np.meshgrid(x, y, indexing="xy")
np.fromiter(gen, dtype=float) ; np.frombuffer(b)
```

`a.shape` `a.ndim` `a.size`	dimensions
`a.dtype` `a.itemsize` `a.nbytes`	element type and memory
`a.reshape(2, -1)`	-1 is inferred; view when possible
`a.T` · `a.ravel()` · `a.flatten()`	view · view if possible · always copy
`np.concatenate` `vstack` `hstack` `stack`	join; `np.split` splits

Indexing: views and copies

Form	Result
`a[1:3, ::2]` slices, ints, `None`, `...`	**view (shares memory)**
`a[[0, 2]]` integer array	**copy**
`a[a > 0]` boolean mask	**copy**
`a.T` · `a.reshape` · `a.view`	**view (reshape may copy)**

```
a[a < 0] = 0                # assignment writes through
b = a[1:3].copy()            # independent data
np.shares_memory(a, b)       # check; b.base is a
np.where(a > 0, a, 0)        # choose per element
np.take(a, idx) ; np.put(a, idx, v)
a[np.ix_([0, 2], [1, 3])]   # outer-product selection
```

Use `&`, `|`, `~` with parentheses for combined masks, never `and` / `or`. Several index arrays broadcast together; one index array on a middle axis moves its result axis.

NumPy 2 renames

Removed	Use
`np.float_`, `np.complex_`	`np.float64`, `np.complex128`
`np.NaN`, `np.Inf`, `np.infty`	`np.nan`, `np.inf`
`np.string_`, `np.unicode_`	`np.bytes_`, `np.str_`
`np.product`, `np.cumproduct`, `np.round_`	`np.prod`, `np.cumprod`, `np.round`
`np.alltrue`, `np.sometrue`	`np.all`, `np.any`
`np.trapz`	`np.trapezoid`
`np.in1d`	`np.isin` (keeps input shape)
`np.row_stack`	`np.vstack`
`np.asfarray`	`np.asarray(x, dtype=float)`
`np.array(x, copy=False)`	`np.asarray(x)` ; `copy=False` now raises if a copy is needed
`np.core`	private (`np._core`); use public names
`np.random.seed`	`np.random.default_rng(seed)`

Lint old code with Ruff rule `NPY201`.

Dtypes and promotion (NEP 50)

Bool	`bool`
Signed / unsigned	`int8 int16 int32 int64` · `uint8 … uint64`
Float	`float16 float32 float64` (`longdouble` platform-specific)
Complex	`complex64 complex128`
Text / time	`StringDType (dtype="T")`, `datetime64[ns]`, `timedelta64`
Records	structured dtypes, `np.dtype([("x", "f8")])`

- The result type depends on operand *types*, never on values.
- Python `int`, `float`, `complex` are **weak**: `float32 array + 3.0` stays `float32`; NumPy scalars are **strong**.
- Python int out of range for the array dtype raises `OverflowError`; array arithmetic wraps silently: `np.int8(100) + np.int8(100)` overflows.
- `int64 + uint64` gives `float64`. Default integer is 64-bit on 64-bit Windows too; default float is `float64`.

```
np.result_type(np.int8, np.float32)  # float32
np.promote_types("i4", "f4")         # float64
a.astype(np.float32, copy=False)     # cast
np.finfo(np.float32).eps ; np.iinfo(np.int8).max
```

Broadcasting

Compare shapes from the **right**; each pair of axes must be equal or one of them 1 (a missing axis counts as 1).

```
(5, 3) + (3,)    -> (5, 3)
(5, 1) + (1, 4)  -> (5, 4)   # outer sum
(5, 3) + (5,)    -> error    # use a[:, None]
x[:, None] - x[None, :]      # pairwise differences
```

Stretching is virtual (no copy of the inputs); the output is a full array. Use `np.broadcast_to` for a read-only view, `keepdims=True` to keep an axis for the next broadcast.

ufuncs and reductions

Math	`add subtract multiply divide power mod`, `exp log log1p sqrt`, `sin cos`
Compare / logic	`greater equal isclose isnan isfinite logical_and`
Rounding	`np.round floor ceil trunc`, `clip(a, lo, hi)`
Methods	`np.add.reduce` `.accumulate` `.outer` `.at`
Options	`out=`, `where=`, `dtype=`

```
a.sum(axis=0)          # collapse rows: one value per column
a.mean(axis=1, keepdims=True)
a.std(ddof=1)          # sample std; default ddof=0
a.argmax(axis=0) ; np.nanmean(a) ; np.cumsum(a)
np.percentile(a, [25, 75]) ; np.histogram(a, bins=10)
np.sort(a, axis=-1) ; np.argsort(a) ; np.searchsorted(s, v)
np.unique(a, return_counts=True)
```

`axis` names the axis that disappears. NaN propagates unless you use the `nan*` versions. Use `with np.errstate(divide="ignore"):` to control floating-point warnings.

Linear algebra

```
A @ B ; np.matmul(A, B)    # matrix product
a * b                      # elementwise, NOT matrix
np.dot(u, v) ; np.vecdot(u, v)
np.linalg.solve(A, b)      # prefer over inv(A) @ b
np.linalg.lstsq(A, b)      # least squares
np.linalg.inv(A) ; np.linalg.det(A)
w, V = np.linalg.eig(A)    # eigh for symmetric
U, s, Vt = np.linalg.svd(A)
np.linalg.norm(x) ; np.linalg.matrix_rank(A)
np.linalg.qr(A) ; np.linalg.cholesky(A)
np.einsum("ij,jk->ik", A, B)  # general contraction
```

Functions broadcast over leading axes (stacks of matrices). `A.mT` transposes the last two axes. Check `np.linalg.cond(A)` before inverting; use `np.allclose`, never `==`, to compare floats.

Random: default_rng

```
rng = np.random.default_rng(42)
rng.random(3)                  # uniform [0, 1)
rng.integers(0, 10, size=5)    # high exclusive
rng.normal(loc=0, scale=1, size=100)
rng.choice(a, size=3, replace=False)
rng.permutation(10) ; rng.shuffle(a)
ss = np.random.SeedSequence(42)
kids = ss.spawn(4)             # independent streams
```

Pass the Generator around; one seed makes a run repeatable. Parallel workers: spawn child seeds, never reuse the same seed. Legacy `np.random.seed` and `RandomState` are frozen.

File input and output

Need	Call
One array, binary	`np.save("a.npy", a)` · `np.load("a.npy")`
Several arrays	`np.savez` · `np.savez_compressed` (`.npz`)
Text, clean	`np.loadtxt(f, delimiter=",", skiprows=1)` · `np.savetxt`
Text with gaps	`np.genfromtxt(f, delimiter=",", filling_values=0)`
Raw bytes	`a.tofile` · `np.fromfile` · `np.memmap`

`np.load` refuses pickled object arrays unless `allow_pickle=True`; never enable it for untrusted files. A `.npy` keeps dtype and shape; text loses them.

Pitfalls

- `ravel` / `reshape` may share memory; a slice assignment changes the parent.
- `a == b` on arrays gives an array; use `np.array_equal` or `np.allclose`.
- `np.float32` sums over many items can lose precision; pass `dtype=np.float64`.
- `np.matrix` is discouraged: use 2-D arrays and `@`.
- Use NumPy functions on arrays, not Python loops: vectorise.